CSE225 — Binary Search Trees, Heaps & Priority Queues

Theory / Written Question Bank with Model Answers | Lecture 11, Lecture 12 and Lecture 13
Definitions · explain-the-logic · write-the-function · time complexity · dry runs · condition-based coding

How to use: cover the answer, write yours on paper, then compare. Code answers are exactly the code style used in the slides (`typedef char ItemType;`, `TreeNode` with `info/left/right`, helper functions outside the class plus a thin member function that passes `root`).

Part A — Definitions and Short Answers

A1. What motivated the introduction of the binary search tree?

Ans. A sorted list gives fast retrieval, `O(logN)`, but slow insertion, `O(N)`; an unsorted list gives fast insertion, `O(1)`, but slow retrieval, `O(N)`. The BST is introduced to improve `InsertItem` without giving up fast retrieval — it aims at `O(logN)` for both.

A2. Define a *tree*.

Ans. A collection of nodes linked to each other (similar to linked lists) in which each node can have 0 or more children (successor nodes), and each node except the root has exactly one parent (predecessor node). Consequently there is exactly one path from the root to any other node.

A3. Define root, leaf and interior node.

Ans. **Root** ⇒ the node with no parent. **Leaf** ⇒ a node with no child. **Interior** ⇒ any non-leaf node.

A4. Define *level* and *height* of a tree.

Ans. **Level** of a node = the number of ancestors (parent, parent's parent, ...) up to the root, i.e. its distance from the root; the root is at level 0. **Height** of the tree = the number of levels. (Some books instead define height as the highest level in the tree.)

A5. Define a *binary tree*.

Ans. A tree with 0–2 children per node: no child, one left child, one right child, or one left and one right child. Every node is the parent of two smaller trees called the left subtree and the right subtree.

A6. Define a *full* binary tree.

Ans. A binary tree is full if each node is either a leaf or possesses exactly two child nodes.

A7. Define a *complete* binary tree.

Ans. A binary tree is complete if all levels (except possibly the last) are completely full, and the last level has all its nodes to the left side.

A8. Are "full" and "complete" the same thing? Justify with the four possibilities.

Ans. No. All four combinations exist and were shown in the lecture: neither full nor complete; full but not complete; not full but complete; and full and complete. Neither property implies the other.

A9. State the key property of a binary search tree.

Ans. The value at any node is larger than the value of its left child — hence larger than all values in its left subtree — and smaller than or equal to the value of its right child — hence smaller than all values in its right subtree.

A10. Write the Structure clause of the BST specification.

Ans. "The placement of each element in the binary tree must satisfy the binary search property: the value of the key of an element is greater than the value of the key of any element in its left subtree, and less than the value of the key of any element in its right subtree."

A11. Define a *priority queue*.

Ans. A structure in which items can be added and deleted like a queue, but which does not necessarily maintain First In First Out order: items can be added in any order, and the item with the highest priority is always the one deleted.

A12. Write the Structure clause of the priority queue specification.

Ans. "The priority queue is arranged to support access to the highest-priority item."

A13. Define a *heap*, naming both of its properties.

Ans. A heap is a complete binary tree (the **shape property**) in which each element contains a value that is less than or equal to the value of each of its children (min-heap) or greater than or equal to the value of each of its children (max-heap) — the **order property**.

A14. Distinguish min-heap from max-heap.

Ans. Min-heap: every element ≤ each of its children, so the smallest value is at the root. Max-heap: every element ≥ each of its children, so the largest value is at the root.

A15. Two facts follow from the heap definition. State them.

Ans. (i) The shape of all heaps with a given number of elements is the same. (ii) The root node always contains the largest value in the heap (max-heap), and in addition every subtree of a heap is itself a heap.

A16. Define ReheapUp and ReheapDown.

Ans. **ReheapDown**: when the order property is violated by the root node only (all other nodes are in place), move that element down from the root position until it reaches a position where the order property is satisfied. **ReheapUp**: when the order property is violated by the last leaf node only, move that element up in the tree until it ends up in its correct position.

A17. Define inorder, postorder and preorder traversal.

Ans. **Inorder**: visit the nodes in the left subtree of a node, then visit the node, then visit the nodes in its right subtree. **Postorder**: visit the left subtree, then the right subtree, then the node. **Preorder**: visit the node, then the left subtree, then the right subtree.

A18. What does `IsFull()` mean for a linked BST? Why is it not about the shape of the tree?

Ans. It reports whether more memory is available, not whether the tree is "full" in the shape sense. It attempts `location = new TreeNode;`. If the allocation succeeds the node is deleted and `false` is returned; if `std::bad_alloc` is thrown, the heap memory is exhausted and `true` is returned.

A19. What is `OrderType` and where is it used?

Ans. `enum OrderType {PRE_ORDER, IN_ORDER, POST_ORDER};` — it is the parameter type of `ResetTree(OrderType order)` and `GetNextItem(ItemType& item, OrderType order, bool& finished)`, selecting which traversal the iteration follows.

A20. What are `FullPQ` and `EmptyPQ`?

Ans. Two empty exception classes declared in `pqtype.h` as `class FullPQ{};` and `class EmptyPQ{};`. `Enqueue` throws `FullPQ()` when `length == maxItems`; `Dequeue` throws `EmptyPQ()` when `length == 0`.

A21. State the precondition and postcondition of `RetrieveItem`.

Ans. **Pre:** key member of item is initialized. **Post:** if an element someItem has a key matching item's key then found = true and item is a copy of someItem; otherwise found = false and item is unchanged. The tree is unchanged in either case.

A22. State the precondition and postcondition of `InsertItem` and of `DeleteItem`.

Ans. **InsertItem** — Pre: tree is not full, item is not in tree. Post: item is in tree, binary search property is maintained. **DeleteItem** — Pre: key member of item is initialized; one and only one element in tree has a key matching item's key. Post: no element in tree has a key matching item's key.

A23. What are the postconditions of `ResetTree` and `GetNextItem`?

Ans. **ResetTree**: current position is prior to the root of the tree. **GetNextItem**: current position is one position beyond the current position at entry; finished = (current position is last in tree); item is a copy of the element at the current position.

A24. Why is a heap *not* a binary search tree?

Ans. A heap only orders each node against its own children; it says nothing about left versus right. Siblings and cousins may appear in any order, so an inorder traversal of a heap is not sorted, whereas an inorder traversal of a BST is.

Part B — Height, Node-Count and Shape Mathematics

B1. How many nodes can there be at level *i* of a binary tree? Prove it informally.

Ans. At most 2^i, for i = 0, 1, 2, ... Level 0 holds only the root ($2^0 = 1$); every node contributes at most 2 children to the next level, so the bound doubles at each level: $\le 2^0$, $\le 2^1$, $\le 2^2$, ...

B2. Derive the maximum number of nodes in a binary tree of height *h*.

Ans. The levels are 0, 1, 2, ..., (h−1), so

```
N = 2^0 + 2^1 + 2^2 + ... + 2^(h-1)
  = (2^((h-1)+1) - 1) / (2 - 1)
  = 2^h - 1
```

B3. Given N nodes, derive the height, and state the minimum and maximum heights.

Ans. From $N = 2^h - 1$ we get $h = \log_2(N + 1)$. The **minimum** height is ceil($\log_2(N + 1)$), attained when the tree is complete (best case). The **maximum** height is N, attained when every node has exactly one child (worst case, a skewed chain).

B4. For N = 10 nodes, compute the minimum and maximum height and describe the two shapes.

Ans. Minimum height = log(10 + 1) = 3.46 ~ 4, which happens when the tree is complete. Maximum height = 10, which happens when each node has exactly one child, producing a single downward chain.

B5. A binary tree has height 4. What is the largest number of nodes it can contain, and the largest number of nodes at its last level?

Ans. Largest total = $2^4 - 1 = 15$. Last level is level 3, so at most $2^3 = 8$ nodes.

B6. Why does BST search degrade to O(N)?

Ans. Because search cost is proportional to the height. If items arrive already sorted, each new item goes to the same side every time and the tree becomes a chain of height N, so the search walks N nodes just like a linked list.

B7. Explain the "impact of order of insertion on tree height" using the three inputs from the lecture.

Ans. The same seven letters give three different trees. Input D B F A C E G gives a short, well-balanced tree (D at the root, B and F as children). Input B A D C G F E gives a taller, lopsided tree. Input A B C D E F G, already sorted, degenerates into a single chain A→B→C→D→E→F→G. The set of keys is identical; only the insertion order differs, and it decides the height and therefore the performance.

B8. Why can a heap always guarantee O(logN) while a BST cannot?

Ans. The heap enforces the shape property: it is always a complete binary tree, so its height is always about $\log_2 N$ and ReheapUp/ReheapDown can never walk more than that. A BST has no shape constraint, so insertion order can make its height as large as N.

Part C — Explain the Logic of a Function

C1. Explain the logic of the recursive `Insert` function, naming its base case and its two general cases.

Ans. **Base case** — `tree == NULL`: the insertion place has been found, so a node is allocated, its two links are set to NULL, and `info` is set to the item. **General case 1** — `item < tree->info`: the item belongs in the left subtree, so recurse on `tree->left`. **General case 2** — otherwise: recurse on `tree->right`. Each recursive call shrinks the problem to one subtree, so the recursion terminates at an empty subtree, which is exactly where a new leaf must be attached.

C2. Why is the first parameter of `Insert` declared `TreeNode *&tree` and not `TreeNode *tree`?

Ans. Because the base case executes `tree = new TreeNode;`. If the pointer were passed by value, the function would modify only its local copy and the new node would never be attached to the parent's `left`/`right` link (or to `root`), so the tree would not change and the memory would leak. Passing a reference to the pointer makes that assignment write directly into the caller's link field. The lecture demonstrates the difference with `void f(int *addr)` (y unchanged) versus `void f(int *&addr)` (y changed).

C3. Explain the logic of `CountNodes`.

Ans. The number of nodes is computed recursively. Base case: an empty tree has length 0. General case: the length equals the number of nodes in the left subtree plus the number of nodes in the right subtree plus 1 for the current node. The recursion therefore folds the answer up from the leaves; the root receives the total.

C4. Explain the logic of `Retrieve`, naming its two base cases and two general cases.

Ans. **Base case 1** — the tree is empty: the item is not in the tree, so `found = false`. **Base case 2** — item equals the current node's info: the item is found, so `item = tree->info; found = true;`. **General case 1** — item < current info: search the left subtree. **General case 2** — item > current info: search the right subtree. The binary search property guarantees that only one side ever has to be searched, which is what makes the search logarithmic on a balanced tree.

C5. Explain the logic of `PrintTree` and say why the output is sorted.

Ans. Base case: if the tree is empty there is nothing to print. General case: print the items in the left subtree, then print the item at the current node, then print the items in the right subtree. Because a BST keeps all smaller keys on the left and all larger keys on the right, this left-node-right order (inorder) emits keys in ascending order.

C6. Explain the three deletion cases of a BST node.

Ans. **Leaf node**: simply detach it — set the parent's link (i.e. `tree`) to NULL and free the node. **Node with one child**: bypass it — set `tree` to that single child, then free the old node, so the child is adopted by the grandparent. **Node with two children**: it cannot simply be removed without breaking the structure, so replace its value with its predecessor — the rightmost node of its left subtree — and then recursively delete that predecessor from the left subtree. The predecessor is the largest key still smaller than everything in the right subtree, so the binary search property is preserved.

C7. Explain the logic of `GetPredecessor`.

Ans. It is called on the *left subtree* of the node being deleted. Since the largest value of a subtree is at its rightmost position, the function simply walks right as far as it can — `while (tree->right != NULL) tree = tree->right;` — and copies the value it lands on into `data`. Note the parameter is a plain `TreeNode*` because the tree is only read, while `data` is a reference because it is the output.

C8. Explain the logic of the recursive `Delete` function.

Ans. It is a search that ends in a removal. If `item < tree->info` the target must be in the left subtree, so recurse there; if `item > tree->info` recurse on the right subtree; otherwise the current node *is* the target and `DeleteNode(tree)` is called to perform the actual removal. Because `tree` is a reference to a pointer, `DeleteNode` can rewrite the parent's link.

C9. Explain the logic of `ReheapUp(root, bottom)`.

Ans. The element sitting at index `bottom` may be too large for its position. While `bottom > root`, compute its parent as `(bottom-1)/2`. If the parent is smaller than the child, the order property is violated, so swap them and repeat the process from the parent's position by calling `ReheapUp(root, parent)`. If the parent is not smaller, the element is already in its correct position and the recursion stops.

C10. Explain the logic of `ReheapDown(root, bottom)`.

Ans. Compute `leftChild = root*2+1` and `rightChild = root*2+2`. If `leftChild > bottom` the node is a leaf and there is nothing to do. If `leftChild == bottom` it is the only child, so it is the `maxChild`. Otherwise both children exist and `maxChild` is the larger of the two. If `elements[root] < elements[maxChild]` the order property is violated, so swap the root with maxChild and continue sinking with `ReheapDown(maxChild, bottom)`. Swapping with the *larger* child is essential: swapping with the smaller one would leave the new parent below one of its children.

C11. Explain the logic of `Enqueue` in the heap-based priority queue.

Ans. If `length == maxItems` the queue is full and `FullPQ()` is thrown. Otherwise `length` is incremented and the new item is written to `items.elements[length-1]`, which is exactly the next free position — the last leaf of the complete tree. Placing it there keeps the shape property but may break the order property at that one leaf, which is precisely the situation ReheapUp is designed for, so `items.ReheapUp(0, length-1)` is called to float the item up to its correct place.

C12. Explain the logic of `Dequeue`.

Ans. If `length == 0`, `EmptyPQ()` is thrown. Otherwise the root, `items.elements[0]`, holds the highest-priority item, so it is copied into `item`. The last leaf, `items.elements[length-1]`, is then moved into the root position; this preserves the complete shape but may break the order property at the root only, which is what ReheapDown fixes. `length` is decremented so the old last slot leaves the tree, and `items.ReheapDown(0, length-1)` sinks the new root into place.

C13. Why must the decrement of `length` happen *before* ReheapDown in Dequeue?

Ans. The value that used to sit in the last slot has already been copied to the root; that slot is no longer part of the heap. If `bottom` still included it, ReheapDown could compare against or swap into a stale element that is logically outside the heap and could pull a removed value back in.

C14. Why does `Retrieve` take `TreeNode* tree` while `Insert` and `Delete` take `TreeNode*& tree`?

Ans. Retrieve never changes the structure of the tree — it only reads `info` and follows links — so a copy of the pointer is sufficient. Insert and Delete both rewrite a link in the parent (attaching a new node, or bypassing / removing an existing one), so they need a reference to the caller's pointer.

C15. Explain how the traversal iterator (`ResetTree` + `GetNextItem`) works.

Ans. The class stores three queues, `QueType preQue, inQue, postQue`. `ResetTree(order)` switches on the order and runs the corresponding recursive traversal (`PreOrder`, `InOrder` or `PostOrder`), which does not print anything but enqueues each visited `info` into the matching queue. The whole traversal is therefore precomputed. `GetNextItem(order, finished)` then simply dequeues one item from that queue, and sets `finished = true` when the queue has become empty, so the caller knows the iteration is over.

C16. In the three traversal functions, only the position of one statement changes. Explain.

Ans. All three recurse on `tree->left` and `tree->right` under `if (tree != NULL)`. The single `Enqueue(tree->info)` statement moves: **before** both calls gives preorder, **between** them gives inorder, and **after** both gives postorder. That one line's position is the entire difference between the three traversals.

Part D — Time Complexity

D1. Complete the sorted vs unsorted list table given at the start of Lecture 11.

	Unsorted List	Sorted List
`RetrieveItem`	O(N)	O(logN)
`InsertItem`	O(1)	O(N)

D2. Give the complexity of `TreeType()`, `IsEmpty()` and `IsFull()`, with reasons.

Ans. All three are **O(1)**. The constructor is a single assignment `root = NULL;`; `IsEmpty` is one pointer comparison; `IsFull` performs one allocation attempt and one deallocation, independent of how many nodes the tree holds.

D3. Give the worst, best and average complexity of `InsertItem`, and say when each occurs.

Ans. Worst **O(N)** — a skewed tree, where insertion walks the whole chain. Best **O(1)** — the insertion place is found immediately (e.g. an empty tree). Average **O(lgN)** — a reasonably balanced tree, where the path length is the height.

D4. Give the worst, best and average complexity of `RetrieveItem`, and say when each occurs.

Ans. Worst **O(N)** — skewed tree, target at the far end (or absent). Best **O(1)** — the target is at the root. Average **O(logN)** — the search follows one root-to-node path whose length is the height.

D5. What is the complexity of `LengthIs` and why is it not O(logN)?

Ans. **O(N)**. Counting cannot use the search property to discard a subtree — every node has to be visited and counted, so both subtrees are always entered.

D6. What is the complexity of `Print`? Why?

Ans. **O(N)** — the traversal must reach every node exactly once to print it.

D7. Give the complexity of ReheapUp, ReheapDown and Swap, with reasons.

Ans. `Swap` is **O(1)** — three assignments. `ReheapUp` and `ReheapDown` are **O(logN)** — each recursive step climbs or sinks exactly one level, and since a heap is a complete binary tree its height is about $\log_2 N$.

D8. Give the complexity of every `PQType` operation.

Operation	Complexity	Reason
`PQType(int max)`	O(1)	3 assignments plus one array allocation
`~PQType()`	O(1)	one `delete []`
`MakeEmpty()`	O(1)	`length = 0;`
`IsFull()`	O(1)	one comparison
`IsEmpty()`	O(1)	one comparison
`Enqueue()`	O(logN)	dominated by ReheapUp
`Dequeue()`	O(logN)	dominated by ReheapDown

D9. Summarise the complexity of every BST operation covered in the two lectures.

Operation	Best	Average	Worst
`TreeType()` / `IsEmpty` / `IsFull`	O(1)	O(1)	O(1)
`InsertItem`	O(1)	O(lgN)	O(N)
`RetrieveItem`	O(1)	O(logN)	O(N)
`LengthIs`	O(N)	O(N)	O(N)
`Print`	O(N)	O(N)	O(N)

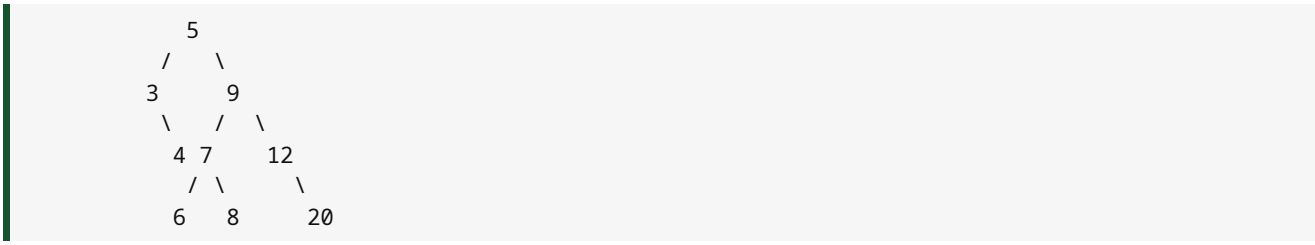
D10. A priority queue is implemented (i) as an unsorted list and (ii) as a heap. Compare Dequeue.

Ans. With an unsorted list the highest-priority item must be located by scanning, which is O(N). With the heap it is already at index 0, so removing it costs only the ReheapDown that follows, i.e. O(logN). This is exactly why the lecture implements the priority queue on top of a heap.

Part E — Dry Runs and Diagrams

E1. Insert 5, 9, 7, 3, 8, 12, 6, 4, 20 into an empty BST in that order. Describe the resulting tree.

Ans. Root 5. Left subtree: 3, whose right child is 4. Right subtree: 9, whose left child is 7 and right child is 12; 7 has children 6 and 8; 12 has right child 20.



E2. For the tree of E1, give the output of `Print()` and of `LengthIs()`.

Ans. `Print()` → 3 4 5 6 7 8 9 12 20. `LengthIs()` → 9.

E3. Trace `InsertItem(7)` into the tree {5 (root), 9 (right child)}. List the calls and the case used by each.

Ans. `InsertItem(7)` → `Insert(root, 7)`: 7 > 5 ⇒ **General case 2** → `Insert(root->right, 7)`: 7 < 9 ⇒ **General case 1** → `Insert(root->right->left, 7)`: pointer is NULL ⇒ **Base case**, so a new node holding 7 is created and attached as the left child of 9.

E4. Trace `RetrieveItem` searching for 8 in the tree of E1.

Ans. At 5: 8 > 5 ⇒ general case 2, go right. At 9: 8 < 9 ⇒ general case 1, go left. At 7: 8 > 7 ⇒ general case 2, go right. At 8: match ⇒ base case 2, `item = 8; found = true;`

E5. Trace `RetrieveItem` searching for 10 in the tree of E1.

Ans. At 5: 10 > 5 ⇒ right. At 9: 10 > 9 ⇒ right. At 12: 10 < 12 ⇒ left. That link is NULL ⇒ base case 1, so `found = false` and item is unchanged.

E6. Trace `CountNodes` on the tree of E1 (report the value returned by each subtree).

Ans. Leaves 4, 6, 8, 20 each return 1 (0 + 0 + 1). Node 3 returns 0 + 1 + 1 = 2. Node 7 returns 1 + 1 + 1 = 3. Node 12 returns 0 + 1 + 1 = 2. Node 9 returns 3 + 2 + 1 = 6. Root 5 returns 2 + 6 + 1 = **9**.

E7. For the tree rooted at 9 with left child 3 (right child 4) and right child 16 (children 10 and 18; 10 has right child 14; 14 has left child 13; 13 has left child 12; 18 has right child 20), give the inorder, postorder and preorder traversals.

Ans. **Inorder:** 3 4 9 10 12 13 14 16 18 20
Postorder: 4 3 12 13 14 10 20 18 16 9
Preorder: 9 3 4 16 10 14 13 12 18 20

E8. In the tree of E7, delete 20, then 18, then 9, then 16. Explain each case.

Ans. **Delete 20** — leaf: 18's right link is set to NULL and the node is freed.
Delete 18 — one child (20): 16's right link is redirected to 20, then the node holding 18 is freed.
Delete 9 — two children: its predecessor is the rightmost node of its left subtree, i.e. 4, so the root becomes 4 and 4 is then deleted from the left subtree.
Delete 16 — two children: the rightmost node of 16's left subtree (10 → 14) is 14, so 16 is replaced by 14 and the old 14 is deleted from that subtree (it has one child 13, which is bypassed).

E9. Write the array representation of the heap 11, 9, 4, 7, 8, 3, 1, 2, 5, 6 and identify the children of index 1 and the parent of index 6.

Ans.

Index	0	1	2	3	4	5	6	7	8	9
Value	11	9	4	7	8	3	1	2	5	6

Children of index 1 are 2(1)+1 = 3 and 2(1)+2 = 4, i.e. 7 and 8. Parent of index 6 is (6−1)/2 = 2, i.e. the value 4.

E10. Trace `Enqueue(15)` on the heap of E9.

Ans. length becomes 11, so 15 is written at index length−1 = 10, as the last leaf. ReheapUp: parent of 10 is (10−1)/2 = 4 holding 8; 8 < 15 so swap — 15 moves to index 4. Parent of 4 is 1 holding 9; 9 < 15 so swap — 15 moves to index 1. Parent of 1 is 0 holding 11; 11 < 15 so swap — 15 becomes the root. Final array: 15 11 4 7 9 3 1 2 5 6 8.

E11. Trace `Dequeue` on the heap of E9.

Ans. `item = elements[0] = 11`, the highest-priority value, is returned. The last leaf 6 (index 9) is copied to index 0 and `length` drops from 10 to 9. ReheapDown(0, 8): children of 0 are 9 and 4, maxChild = 9; 6 < 9 so swap — 6 moves to index 1. Children of 1 are 7 and 8, maxChild = 8; 6 < 8 so swap — 6 moves to index 4. Index 4 has no child within bound 8, so it stops. Final array: 9 8 4 7 6 3 1 2 5.

E12. ReheapDown is applied to the max-heap whose root is 6 and whose remaining nodes are 11, 4 / 7, 9, 3, 1 / 2, 5, 8. Trace it.

Ans. Root 6 vs children 11 and 4: maxChild = 11, and 6 < 11, so swap — 11 becomes the root and 6 sinks to 11's old place. Its children are now 7 and 9: maxChild = 9, and 6 < 9, so swap. 6 now sits above 8: maxChild = 8, and 6 < 8, so swap. 6 is now a leaf and the heap is restored: 11; 9, 4; 7, 8, 3, 1; 2, 5, 6.

E13. ReheapUp is applied because the last leaf of the max-heap 9; 8, 4; 7, 6, 3, 1; 2, 5, 11 is 11. Trace it.

Ans. 11 is at index 9; parent (9−1)/2 = 4 holds 6, and 6 < 11, so swap. 11 is now at index 4; parent (4−1)/2 = 1 holds 8, and 8 < 11, so swap. 11 is now at index 1; parent 0 holds 9, and 9 < 11, so swap. 11 becomes the root: 11; 9, 4; 7, 8, 3, 1; 2, 5, 6.

E14. Draw a binary tree that is (i) full but not complete and (ii) complete but not full.

Ans. (i) Full but not complete — every node has 0 or 2 children, but level 2 is not filled left-to-right:

```
      A
     / \
    B   C
       / \
      D   E
     / \
    F   G
```

(ii) Complete but not full — levels fill left to right, but node C has only one child:

```
      A
     / \
    B   C
   / \  /
  D  E F
```

E15. Given the preorder 9 3 4 16 10 14 13 12 18 20 and the inorder 3 4 9 10 12 13 14 16 18 20, identify the root and its two subtrees.

Ans. The first element of the preorder, 9, is the root. In the inorder it splits the sequence: everything before it, 3 4, is the left subtree, and everything after it, 10 12 13 14 16 18 20, is the right subtree. (Applying the same rule recursively rebuilds the whole tree.)

Part F — Write the Code (exactly as developed in the lectures)

F1. Write the `TreeNode` structure used for a binary search tree.

Ans.

```
struct TreeNode
{
  ItemType info;
  TreeNode *left;
  TreeNode *right;
};
```

A node is drawn as three fields: `left | info | right`.

F2. Write `binarysearchtree.h`.

Ans.

```
#ifndef BINARYSEARCHTREE_H_INCLUDED
#define BINARYSEARCHTREE_H_INCLUDED
typedef char ItemType;
struct TreeNode{ItemType info; TreeNode *left, *right;};
enum OrderType {PRE_ORDER, IN_ORDER, POST_ORDER};
class TreeType
{
public:
  TreeType();
  ~TreeType();
  void MakeEmpty();
  bool IsEmpty();
  bool IsFull();
  int  LengthIs();
  void RetrieveItem(ItemType& item, bool& found);
  void InsertItem(ItemType item);
  void DeleteItem(ItemType item);
  void ResetTree(OrderType order);
  void GetNextItem(ItemType& item, OrderType order, bool& finished);
  void Print();
private:
  TreeNode* root;
};
#endif // BINARYSEARCHTREE_H_INCLUDED
```

F3. Write the constructor and `IsEmpty`.

Ans.

```
TreeType::TreeType()
{
  root = NULL;                 // O(1)
}

bool TreeType::IsEmpty()
{
  return root == NULL;         // O(1)
}
```

F4. Write `IsFull` for the linked BST.

Ans.

```
bool TreeType::IsFull()
{
  TreeNode* location;
  try
  {
    location = new TreeNode;
    delete location;
    return false;
  }
  catch(std::bad_alloc exception)
  {
    return true;
  }
}                              // O(1)
```

Requires `#include <new>`.

F5. Write `InsertItem` together with its recursive helper.

Ans.

```
void Insert(TreeNode *&tree, ItemType item)
{
  if (tree == NULL)            // Base case: insertion place found
  {
    tree = new TreeNode;
    tree->right = NULL;
    tree->left  = NULL;
    tree->info  = item;
  }
  else if (item < tree->info)
    Insert(tree->left, item);  // General case 1: insert in left subtree
  else
    Insert(tree->right, item); // General case 2: insert in right subtree
}

void TreeType::InsertItem(ItemType item)
{
  Insert(root, item);
}
```

Worst O(N), best O(1), average O(lgN).

F6. Write `LengthIs` together with `CountNodes`.

Ans.

```
int CountNodes(TreeNode* tree)
{
  if (tree == NULL)
    return 0;
  else
    return CountNodes(tree->left) + CountNodes(tree->right) + 1;
}

int TreeType::LengthIs()
{
  return CountNodes(root);
}                              // O(N)
```

F7. Write `RetrieveItem` together with `Retrieve`.

Ans.

```
void Retrieve(TreeNode* tree, ItemType& item, bool& found)
{
    if (tree == NULL)
        found = false;
    else if (item < tree->info)
        Retrieve(tree->left, item, found);
    else if (item > tree->info)
        Retrieve(tree->right, item, found);
    else
    {
        item  = tree->info;
        found = true;
    }
}

void TreeType::RetrieveItem(ItemType& item, bool& found)
{
    Retrieve(root, item, found);
}
```

Worst O(N), best O(1), average O(logN).

F8. Write `Print` together with `PrintTree`.

Ans.

```
void PrintTree(TreeNode* tree)
{
    if (tree != NULL)
    {
        PrintTree(tree->left);
        cout << tree->info << endl;
        PrintTree(tree->right);
    }
}

void TreeType::Print()
{
    PrintTree(root);
}                               // O(N)
```

F9. Write `DeleteItem` and the recursive `Delete`.

Ans.

```
void DeleteNode(TreeNode*& tree);
void GetPredecessor(TreeNode* tree, ItemType& data);

void TreeType::DeleteItem(ItemType item)
{
    Delete(root, item);
}

void Delete(TreeNode*& tree, ItemType item)
{
    if (item < tree->info)
        Delete(tree->left, item);
    else if (item > tree->info)
        Delete(tree->right, item);
    else
        DeleteNode(tree);
}
```

F10. Write `DeleteNode`, covering all cases.

Ans.

```
void DeleteNode(TreeNode*& tree)
{
    ItemType data;
    TreeNode* tempPtr;

    tempPtr = tree;
    if (tree->left == NULL && tree->right == NULL)   // leaf
    {
        tree = NULL;
        delete tempPtr;
    }
    else if (tree->left == NULL)                     // only right child
    {
        tree = tree->right;
        delete tempPtr;
    }
    else if (tree->right == NULL)                    // only left child
    {
        tree = tree->left;
        delete tempPtr;
    }
    else                                            // two children
    {
        GetPredecessor(tree->left, data);
        tree->info = data;
        Delete(tree->left, data);
    }
}
```

F11. Write `GetPredecessor`.

Ans.

```
void GetPredecessor(TreeNode* tree, ItemType& data)
{
    while (tree->right != NULL)
        tree = tree->right;
    data = tree->info;
}
```

It is always called as `GetPredecessor(tree->left, data)`, so it returns the largest key of the left subtree.

F12. Write the three traversal helpers that fill the queues.

Ans.

```
void InOrder(TreeNode* tree, QueType& inQue)
{
  if (tree != NULL)
  {
    InOrder(tree->left, inQue);
    inQue.Enqueue(tree->info);
    InOrder(tree->right, inQue);
  }
}
void PostOrder(TreeNode* tree, QueType& postQue)
{
  if (tree != NULL)
  {
    PostOrder(tree->left, postQue);
    PostOrder(tree->right, postQue);
    postQue.Enqueue(tree->info);
  }
}
void PreOrder(TreeNode* tree, QueType& preQue)
{
  if (tree != NULL)
  {
    preQue.Enqueue(tree->info);
    PreOrder(tree->left, preQue);
    PreOrder(tree->right, preQue);
  }
}
```

F13. Write `ResetTree`.

Ans.

```
void TreeType::ResetTree(OrderType order)
{
  switch (order)
  {
    case PRE_ORDER : PreOrder(root, preQue);
                     break;
    case IN_ORDER  : InOrder(root, inQue);
                     break;
    case POST_ORDER: PostOrder(root, postQue);
                     break;
  }
}
```

The class holds `QueType preQue, inQue, postQue;` as private members alongside `TreeNode* root;`.

F14. Write `GetNextItem`.

Ans.

```
ItemType TreeType::GetNextItem(OrderType order, bool& finished)
{
  finished = false;
  ItemType item;
  switch (order)
  {
    case PRE_ORDER  : preQue.Dequeue(item);
                      if (preQue.IsEmpty())
                        finished = true;
                      break;
    case IN_ORDER   : inQue.Dequeue(item);
                      if (inQue.IsEmpty())
                        finished = true;
                      break;
    case POST_ORDER : postQue.Dequeue(item);
                      if (postQue.IsEmpty())
                        finished = true;
                      break;
  }
  return item;
}
```

Note on the slides: the header (Lecture 11) declares `void GetNextItem(ItemType& item, OrderType order, bool& finished);` — returning the item through a reference parameter — while the implementation slide (Lecture 12) writes it as `ItemType GetNextItem(OrderType order, bool& finished)`, returning the item. Both appear in the lectures; quote whichever form the question asks for, and keep the two consistent in your own code.

F15. Write the `HeapType` class declaration.

Ans.

```
template<class ItemType>
class HeapType
{
  void ReheapDown(int root, int bottom);
  void ReheapUp(int root, int bottom);
  ItemType* elements;
  int numElements;
};
```

F16. Write `Swap` and `ReheapUp`.

Ans.

```
template <class ItemType>
void Swap(ItemType& one, ItemType& two)
{
  ItemType temp;
  temp = one;
  one  = two;
  two  = temp;
}                                       // O(1)

template<class ItemType>
void HeapType<ItemType>::ReheapUp(int root, int bottom)
{
  int parent;
  if (bottom > root)
  {
    parent = (bottom-1) / 2;
    if (elements[parent] < elements[bottom])
    {
      Swap(elements[parent], elements[bottom]);
      ReheapUp(root, parent);
    }
  }
}                                       // O(logN)
```

F17. Write `ReheapDown`.

Ans.

```
template<class ItemType>
void HeapType<ItemType>::ReheapDown(int root, int bottom)
{
  int maxChild, rightChild = root*2+2, leftChild = root*2+1;
  if (leftChild <= bottom)              // there is at least one child
  {
    if (leftChild == bottom)            // it is the only child
      maxChild = leftChild;
    else                                // there are two children
    {
      if (elements[leftChild] <= elements[rightChild])
        maxChild = rightChild;
      else
        maxChild = leftChild;
    }
    if (elements[root] < elements[maxChild])
    {
      Swap(elements[root], elements[maxChild]);
      ReheapDown(maxChild, bottom);
    }
  }
}                                       // O(logN)
```

F18. Write `pqtype.h`.

Ans.

```
#ifndef PQTYPE_H_INCLUDED
#define PQTYPE_H_INCLUDED

class FullPQ{};
class EmptyPQ{};
#include "heap.h"
template<class ItemType>
class PQType
{
public:
  PQType(int);
  ~PQType();
  void MakeEmpty();
  bool IsEmpty();
  bool IsFull();
  void Enqueue(ItemType newItem);
  void Dequeue(ItemType& item);
private:
  int length;
  HeapType<ItemType> items;
  int maxItems;
};
#endif // PQTYPE_H_INCLUDED
```

F19. Write the PQType constructor, destructor and `MakeEmpty`.

Ans.

```
template<class ItemType>
PQType<ItemType>::PQType(int max)
{
  maxItems = max;
  items.elements = new ItemType[max];
  length = 0;
}                                       // O(1)

template<class ItemType>
PQType<ItemType>::~PQType()
{
  delete [] items.elements;
}                                       // O(1)

template<class ItemType>
void PQType<ItemType>::MakeEmpty()
{
  length = 0;
}                                       // O(1)
```

F20. Write `IsFull` and `IsEmpty` for PQType.

Ans.

```
template<class ItemType>
bool PQType<ItemType>::IsFull()
{
  return length == maxItems;            // O(1)
}

template<class ItemType>
bool PQType<ItemType>::IsEmpty()
{
  return length == 0;                   // O(1)
}
```

F21. Write `Enqueue`.

Ans.

```
template<class ItemType>
void PQType<ItemType>::Enqueue(ItemType newItem)
{
  if (length == maxItems)
    throw FullPQ();
  else
  {
    length++;
    items.elements[length-1] = newItem;
    items.ReheapUp(0, length-1);
  }
}                                       // O(logN)
```

F22. Write `Dequeue`.

Ans.

```
template<class ItemType>
void PQType<ItemType>::Dequeue(ItemType& item)
{
  if (length == 0)
    throw EmptyPQ();
  else
  {
    item = items.elements[0];
    items.elements[0] = items.elements[length-1];
    length--;
    items.ReheapDown(0, length-1);
  }
}                                       // O(logN)
```

Part G — Condition-Based Coding (build on the slide functions)

G1. Write a recursive function that returns the height of a binary tree (number of levels).

Ans. Same shape as `CountNodes`, but take the maximum of the two sides instead of the sum:

```
int Height(TreeNode* tree)
{
  if (tree == NULL)
    return 0;
  int l = Height(tree->left);
  int r = Height(tree->right);
  if (l > r) return l + 1;
  else       return r + 1;
}
```

O(N), since every node is visited.

G2. Write a function that counts the number of leaf nodes in a binary tree.

Ans.

```
int CountLeaves(TreeNode* tree)
{
  if (tree == NULL)
    return 0;
  else if (tree->left == NULL && tree->right == NULL)
    return 1;                       // definition of a leaf: no child
  else
    return CountLeaves(tree->left) + CountLeaves(tree->right);
}
```

O(N).

G3. Write a function that counts the interior (non-leaf) nodes of a binary tree.

Ans.

```
int CountInterior(TreeNode* tree)
{
  if (tree == NULL)
    return 0;
  if (tree->left == NULL && tree->right == NULL)
    return 0;                       // a leaf is not interior
  return CountInterior(tree->left) + CountInterior(tree->right) + 1;
}
```

Equivalently `CountNodes(tree) - CountLeaves(tree)`.

G4. Write a function that returns the number of nodes having exactly one child.

Ans.

```
int CountSingleChild(TreeNode* tree)
{
  if (tree == NULL)
    return 0;
  int self = 0;
  if ((tree->left == NULL) != (tree->right == NULL))
    self = 1;                       // exactly one link is NULL
  return self + CountSingleChild(tree->left)
              + CountSingleChild(tree->right);
}
```

G5. Write a function that returns the smallest value stored in a BST, without traversing every node.

Ans. By the binary search property every smaller key lies to the left, so walk left as far as possible — this is the mirror image of `GetPredecessor`:

```
void GetSmallest(TreeNode* tree, ItemType& data)
{
  while (tree->left != NULL)
    tree = tree->left;
  data = tree->info;
}
```

Precondition: the tree is not empty. Cost O(height), i.e. O(logN) average, O(N) worst.

G6. Write a function that returns the largest value stored in a BST.

Ans. Walk right as far as possible — this is exactly `GetPredecessor` applied to the whole tree:

```
void GetLargest(TreeNode* tree, ItemType& data)
{
  while (tree->right != NULL)
    tree = tree->right;
  data = tree->info;
}
```

G7. Write `MakeEmpty` (and the destructor) for `TreeType`, freeing every node.

Ans. The nodes must be freed after their children, so a **postorder** traversal is used:

```
void Destroy(TreeNode*& tree)
{
  if (tree != NULL)
  {
    Destroy(tree->left);
    Destroy(tree->right);
    delete tree;
    tree = NULL;
  }
}

void TreeType::MakeEmpty()
{
  Destroy(root);                 // postcondition: tree exists and is empty
}

TreeType::~TreeType()
{
  Destroy(root);
}
```

O(N). A preorder `delete` would free the node before following its links.

G8. Write a function that prints the BST in descending key order.

Ans. Mirror the inorder traversal: right, node, left.

```
void PrintDescending(TreeNode* tree)
{
  if (tree != NULL)
  {
    PrintDescending(tree->right);
    cout << tree->info << endl;
    PrintDescending(tree->left);
  }
}
```

O(N).

G9. Write a boolean function that reports whether a given item is present in a BST (a simplified RetrieveItem).

Ans.

```
bool Search(TreeNode* tree, ItemType item)
{
  if (tree == NULL)
    return false;                // base case 1
  else if (item < tree->info)
    return Search(tree->left, item);
  else if (item > tree->info)
    return Search(tree->right, item);
  else
    return true;                 // base case 2
}
```

Average O(logN), worst O(N).

G10. Write an iterative (non-recursive) version of BST search.

Ans.

```
bool SearchIterative(TreeNode* tree, ItemType item)
{
  while (tree != NULL)
  {
    if (item < tree->info)
      tree = tree->left;
    else if (item > tree->info)
      tree = tree->right;
    else
      return true;
  }
  return false;
}
```

Same complexity as the recursive version but without the call stack. Note the parameter can stay `TreeNode*` because the tree is never modified.

G11. Write a function that counts how many values in a BST are greater than a given key `k`, skipping subtrees that cannot contain such values.

Ans.

```
int CountGreater(TreeNode* tree, ItemType k)
{
  if (tree == NULL)
    return 0;
  if (tree->info <= k)
    return CountGreater(tree->right, k);       // whole left subtree is <= k
  else
    return 1 + CountGreater(tree->left, k)
             + CountGreater(tree->right, k);   // right subtree is all > k
}
```

The pruning uses the binary search property: if the current key is ≤ k, no key in its left subtree can exceed k.

G12. Write a function that returns the sum of all values of a BST of integers.

Ans. Same recursion skeleton as `CountNodes`, with the value in place of the constant 1:

```
int SumTree(TreeNode* tree)
{
  if (tree == NULL)
    return 0;
  return SumTree(tree->left) + SumTree(tree->right) + tree->info;
}
```

O(N) — every value contributes, so no subtree can be pruned.

G13. Write a function that checks whether a binary tree is a full tree.

Ans. Apply the definition directly: every node is a leaf or has exactly two children.

```
bool IsFullTree(TreeNode* tree)
{
  if (tree == NULL)
    return true;
  if (tree->left == NULL && tree->right == NULL)
    return true;                                  // a leaf
  if (tree->left != NULL && tree->right != NULL)
    return IsFullTree(tree->left) && IsFullTree(tree->right);
  return false;                                   // exactly one child
}
```

G14. Write a function that counts the number of nodes at a given level of a binary tree.

Ans. The root is at level 0, so each recursive step decreases the target level by one:

```
int CountAtLevel(TreeNode* tree, int level)
{
  if (tree == NULL)
    return 0;
  if (level == 0)
    return 1;
  return CountAtLevel(tree->left,  level-1)
       + CountAtLevel(tree->right, level-1);
}
```

The result can never exceed 2^{level}.

G15. Modify `ReheapUp` so that it maintains a min-heap instead of a max-heap.

Ans. Only the comparison changes — swap when the parent is *larger* than the child:

```
template<class ItemType>
void HeapType<ItemType>::ReheapUp(int root, int bottom)
{
  int parent;
  if (bottom > root)
  {
    parent = (bottom-1) / 2;
    if (elements[parent] > elements[bottom])   // was <
    {
      Swap(elements[parent], elements[bottom]);
      ReheapUp(root, parent);
    }
  }
}
```

G16. Modify `ReheapDown` for a min-heap.

Ans. Choose the *smaller* child and swap when the root is larger:

```
template<class ItemType>
void HeapType<ItemType>::ReheapDown(int root, int bottom)
{
  int minChild, rightChild = root*2+2, leftChild = root*2+1;
  if (leftChild <= bottom)
  {
    if (leftChild == bottom)
      minChild = leftChild;
    else
    {
      if (elements[leftChild] <= elements[rightChild])
        minChild = leftChild;                 // pick the smaller one
      else
        minChild = rightChild;
    }
    if (elements[root] > elements[minChild])   // was <
    {
      Swap(elements[root], elements[minChild]);
      ReheapDown(minChild, bottom);
    }
  }
}
```

G17. Using only the heap operations, write a function that returns the largest value of a max-heap without removing it, and give its complexity.

Ans.

```
template<class ItemType>
ItemType PQType<ItemType>::Peek()
{
  if (length == 0)
    throw EmptyPQ();
  return items.elements[0];
}
```

O(1) — by the order property the root always contains the largest value, and the root is stored at index 0.

G18. Given an array of N values, describe how to obtain them in descending order using a priority queue, and give the complexity.

Ans. Enqueue all N values into a max-heap priority queue, then Dequeue N times: each Dequeue returns the current largest value, so the output is in descending order.

```
PQType<int> pq(N);
for (int i = 0; i < N; i++)  pq.Enqueue(a[i]);
for (int i = 0; i < N; i++)  pq.Dequeue(a[i]);   // a[] now descending
```

N enqueues at O(logN) each plus N dequeues at O(logN) each gives O(N logN).

G19. Write a function that, given the index `i` of an element in the heap array, prints the values of its parent and both children (or reports that they do not exist), for a heap of the given `length`.

Ans.

```
void ShowFamily(ItemType elements[], int length, int i)
{
  int parent = (i-1)/2, l = 2*i+1, r = 2*i+2;
  if (i > 0)      cout << "parent: " << elements[parent] << endl;
  else            cout << "no parent (root)" << endl;
  if (l < length) cout << "left  : " << elements[l] << endl;
  else            cout << "no left child"  << endl;
  if (r < length) cout << "right : " << elements[r] << endl;
  else            cout << "no right child" << endl;
}
```

Index arithmetic: root at 0, left child 2i+1, right child 2i+2, parent (i−1)/2.

G20. A tree class must report whether the tree is balanced enough, defined as: the heights of the left and right subtrees of the root differ by at most 1. Write it.

Ans.

```
bool RootBalanced(TreeNode* tree)
{
  if (tree == NULL)
    return true;
  int l = Height(tree->left);
  int r = Height(tree->right);
  int d = l - r;
  if (d < 0) d = -d;
  return d <= 1;
}
```

Uses the `Height` function of G1; O(N).

G21. Write a function that inserts an item into a BST only if it is not already present (the InsertItem precondition says item must not be in the tree — make the code enforce it).

Ans.

```
void InsertUnique(TreeNode*& tree, ItemType item)
{
  if (tree == NULL)                    // base case: place found
  {
    tree = new TreeNode;
    tree->left = NULL;
    tree->right = NULL;
    tree->info = item;
  }
  else if (item < tree->info)
    InsertUnique(tree->left, item);
  else if (item > tree->info)
    InsertUnique(tree->right, item);
  // else: item == tree->info, so do nothing (duplicate rejected)
}
```

The only change from the lecture's `Insert` is that the final `else` becomes `else if (item > tree->info)`, so equal keys fall through and are ignored.

G22. Given the postcondition "Print the values of the tree that lie between two keys `low` and `high` inclusive, in ascending order", write the function.

Ans.

```
void PrintRange(TreeNode* tree, ItemType low, ItemType high)
{
  if (tree != NULL)
  {
    if (tree->info > low)                 // left subtree may hold values ≥ low
      PrintRange(tree->left, low, high);
    if (tree->info >= low && tree->info <= high)
      cout << tree->info << " ";
    if (tree->info < high)                // right subtree may hold values ≤ high
      PrintRange(tree->right, low, high);
  }
}
```

It is an inorder traversal (so the output is ascending) in which the binary search property prunes the subtrees that cannot contain any value of the range.

G23. Explain, with reference to the code, why `Delete` as written in the lecture crashes if the item is not present, and how the precondition prevents it.

Ans. `Delete` has no `tree == NULL` test: it only compares `item` with `tree->info`. If the key is absent, the recursion eventually reaches a NULL pointer and dereferences it, causing a crash. This is exactly why the specification states the precondition "one and only one element in tree has a key matching item's key" — the caller must guarantee presence, for example by calling `RetrieveItem` first. A defensive version would begin with `if (tree == NULL) return;`.